Министерство цифрового развития, связи и массовых коммуникаций РФ
Уральский технический институт связи и информатики (филиал)
ФГБОУ ВО «Сибирский государственный университет
телекоммуникаций и информатики» в г. Екатеринбурге
(УрТИСИ СибГУТИ)

Отчёт
по учебной практике
по МДК 02.02 «Инструментальные средства разработки
программного обеспечения»
на тему «Разработка веб-приложения»

студента IV курса 284 группы

Фамилия Чаукин

Имя, отчество Василий Станиславович

Факультет Инфокоммуникаций, информатики и управления

Специальность 09.02.07 «Информационные системы
и программирование»

Екатеринбург, 2026

Отзыв руководителя

Содержание

Введение..... 4
1 Демонстрационный экзамен..... 4
1.1 Анализ предметной области и технического задания..... 5
1.2 Выбор стека технологий и инструментов разработки..... 6
1.3 Работа с базой данных..... 7
1.4 Практическая реализация решения..... 10
1.5 Интерфейс программы..... 13
2 Дипломный проект..... 16
2.1 Структура приложения..... 16
2.2 Макеты основных страниц приложения..... 17
2.3 Разработка API..... 19
2.4 Инструкция по работе с продуктом..... 21
Заключение..... 28
Библиография..... 29
Приложение А..... 30

Введение

Данная учебная практика направлена на подготовку к демонстрационному экзамену и разработке практической части дипломного проекта.

Цель учебной практики – проработать вариант демонстрационного экзамена, разработать подсистему для работы с продукцией мебельной компании и составить пояснительную записку к практической части дипломного проекта.

Для достижения поставленной цели необходимо решить следующие задачи:

– проанализировать техническое задание и предметную область;

– спроектировать и создать базу данных для хранения информации о продукции и цехах производства;

– подготовить данные для импорта и загрузить их в разработанную базу данных;

– разработать приложение для просмотра, добавления и редактирования продукции компании;

– реализовать функционал отображения списка цехов, задействованных в производстве продукции;

– разработать алгоритм расчета времени изготовления продукции на основе времени прохождения по цехам;

– реализовать метод расчета необходимого количества сырья с учетом потерь;

– выполнить тестирование и отладку разработанного программного продукта;

– разработать документацию к приложению и оформить пояснительную записку к практической части дипломного проекта.

1 Демонстрационный экзамен

1.1 Анализ предметной области

Производственная компания «Комфорт» занимается изготовлением мебели. Компания занимается производством и реализует свою продукцию через партнеров, которые доставляют продукцию компании до конечных потребителей.

С целью оптимизации деятельности компании разрабатывается система,

в которой заказчик выделил несколько подсистем:

− продукция;

− склад и материалы;

− производство;

− сотрудники;

− партнеры.

В данной области определены следующие основные составляющие:

1. Партнеры: тип, наименование компании, юридический адрес, ИНН, ФИО директора, контактные данные (телефон, email), логотип, рейтинг, места продаж, история реализации продукции для последующего расчета скидок.

Партнеры покупают продукцию со скидками, которые зависят от общего объема их продаж. Скидки мотивируют партнеров к большему объему продаж.

Партнеры реализуют товары в розничных и оптовых магазинах, интернет магазинах, другим компаниям по продаже товаров и оказанию услуг.

Взаиморасчеты производственной компании с партнерами осуществляются в рублях.

2. Менеджеры: поиск и регистрация партнеров, решение об изменении рейтинга партнера, ведение истории изменений рейтинга, прием заявок от партнеров, формирование для них предложения на основании статистики истории продаж, полученной от аналитиков компании.

3. Заявка: если партнер ранее производил закупку продукции и предоставил данные о ее реализации, то на основании этих данных и текущих объемах готовой продукции менеджер формирует предложение. Если работа ведется с новым партнером, то менеджер предоставляет каталог готовой продукции и принимает заявку. Затем менеджер должен указать стоимость и дату производства каждой указанной в заявке единицы продукции. После согласования списка продукции с количеством, стоимостью и сроком изготовления заявка считается созданной.

Партнер может отменить заявку до момента внесения предоплаты. Если предоплата не поступает в течение трех дней, то заявка отменяется автоматически, о чем обязательно уведомляется партнер. После поступления предоплаты продукция, указанная в заявке и отсутствующая на складе в нужном количестве, запускается в производство. По готовности всей продукции менеджер должен

предложить организовать доставку. В момент получения продукции (путем доставки или самовывоза) партнер производит полную оплату. После проверки количества и качества продукции партнером заявка считается выполненной.

4. Сотрудники: ФИО, дата рождения, паспортные данные, банковские реквизиты, наличие семьи, состояние здоровья.

5. Кадры: допуск к работе с конкретным оборудованием. Мастер производства занимается технологией производства, контролем качества готовой продукции, заказом материалов у поставщиков.

6. Доступ: двери открываются при использовании карты сотрудника, система турникетов передает данные о перемещении сотрудников в приложение для формирования аналитиками статистики по сотрудникам.

7. Материалы (сырье): тип, наименование, поставщики, количество в упаковке, единица измерения, описание, изображение, стоимость, количество на складе, минимальное допустимое количество, история изменений количества материалов на складе.

8. Склад: регистрация поступлений материалов, резервирование материалов для производства, отпуск и списание материалов со склада в производство, поступление и выбытие готовой продукции, получение информации о текущих остатках и движении товарно-материальных ценностей по складу.

9. Поставщики: тип, наименование, ИНН, история поставок материалов. Менеджеры контролируют список поставщиков, аналитики формируют информацию о качестве поставляемых материалов для принятия решения о прекращении работы с определенными компаниями.

10. Продукция: артикул, тип, наименование, описание, изображение, минимальная стоимость для партнера, размер упаковки (длина, ширина, высота), вес без упаковки, вес с упаковкой, сертификат качества (скан документа), номер стандарта, история изменений минимальной стоимости для партнера, время изготовления, себестоимость, номер цеха, количество человек на производстве, необходимые материалы для производства.

1.2 Выбор стека технологий и инструментов разработки

Для создания данного программного продукта был выбран стек технологий на базе фреймворка fastapi с использованием языка программирования python. Выбор языка python и фреймворка fastapi обусловлен их надежностью, современной архитектурой и удобством разработки прикладных информационных систем. Используемый стек позволяет создавать производительные веб-приложения с поддержкой базы данных, разграничения доступа и обработки пользовательских запросов.

Для реализации пользовательского интерфейса использовалась модель MVC, а для представлений jinja 2. Данный подход позволяет строить веб-приложение на основе модели разделения логики, данных и интерфейса, что делает структуру

проекта более понятной и удобной для сопровождения. Использование контроллеров и представлений позволило реализовать страницы просмотра, добавления и редактирования продукции, материалов, цехов и заявок на добавление продукции.

В качестве системы хранения данных была выбрана postgresql. Эта система управления базами данных удобна тем, что легко подключается к приложению и подходит для учебных и демонстрационных проектов. Использование postgresql позволило организовать хранение сведений о продукции, типах продукции, материалах, цехах, пользователях и заявках без необходимости разворачивать отдельный сервер базы данных.

Для взаимодействия с базой данных применялась библиотека sqlalchemy. Работа с данными организована через SQL-запросы и отдельные классы-модели, отвечающие за доступ к таблицам и выполнение операций чтения, добавления, изменения и удаления данных. Такой подход обеспечивает наглядность логики работы с базой данных и позволяет гибко управлять структурой и содержимым хранимой информации.

Таким образом, выбранный набор технологий позволил реализовать веб-приложение с поддержкой базы данных, аутентификации, ролевой модели доступа и средствами управления данными о продукции мебельной компании. Использованный стек является современным, удобным для сопровождения и хорошо подходит для решения поставленной задачи.

1.3 Работа с базой данных

Перед проектированием базы данных были определены основные сущности предметной области, необходимые для хранения информации о продукции, типах продукции, материалах, цехах, пользователях и заявках на добавление новой продукции. В результате были выделены следующие сущности и их атрибуты:

productType:

- id (PK);
- product_type;
-product_type_coef.

materialType:

- id (PK);
- material_type;

- raw_material_loss_percentage.

workshop:

- id (PK);
- workshop_name;
- workshop_type;
- people_ount.

product:

- id (PK);
- product_name;
- product_type (FK);
- articul;
- min_partner_cost;
- base_material(FK).

ProductWorkshop:

- id (PK);
- product (FK);
- workshop (FK);
- make_time.

Связи между сущностями:

- типы продукции и продукция (один ко многим): один тип продукции может соответствовать нескольким изделиям, при этом каждое изделие относится только к одному типу продукции;
- типы материалов и продукция (один ко многим): один тип материала может использоваться для нескольких изделий, при этом каждое изделие связано только с одним типом материала;
- продукция и цеха (многие ко многим через сущность product_workshop): одно изделие может проходить обработку в нескольких цехах, а один цех может участвовать в изготовлении нескольких изделий;

ER-диаграмма системы представлена на рисунке 1.1.

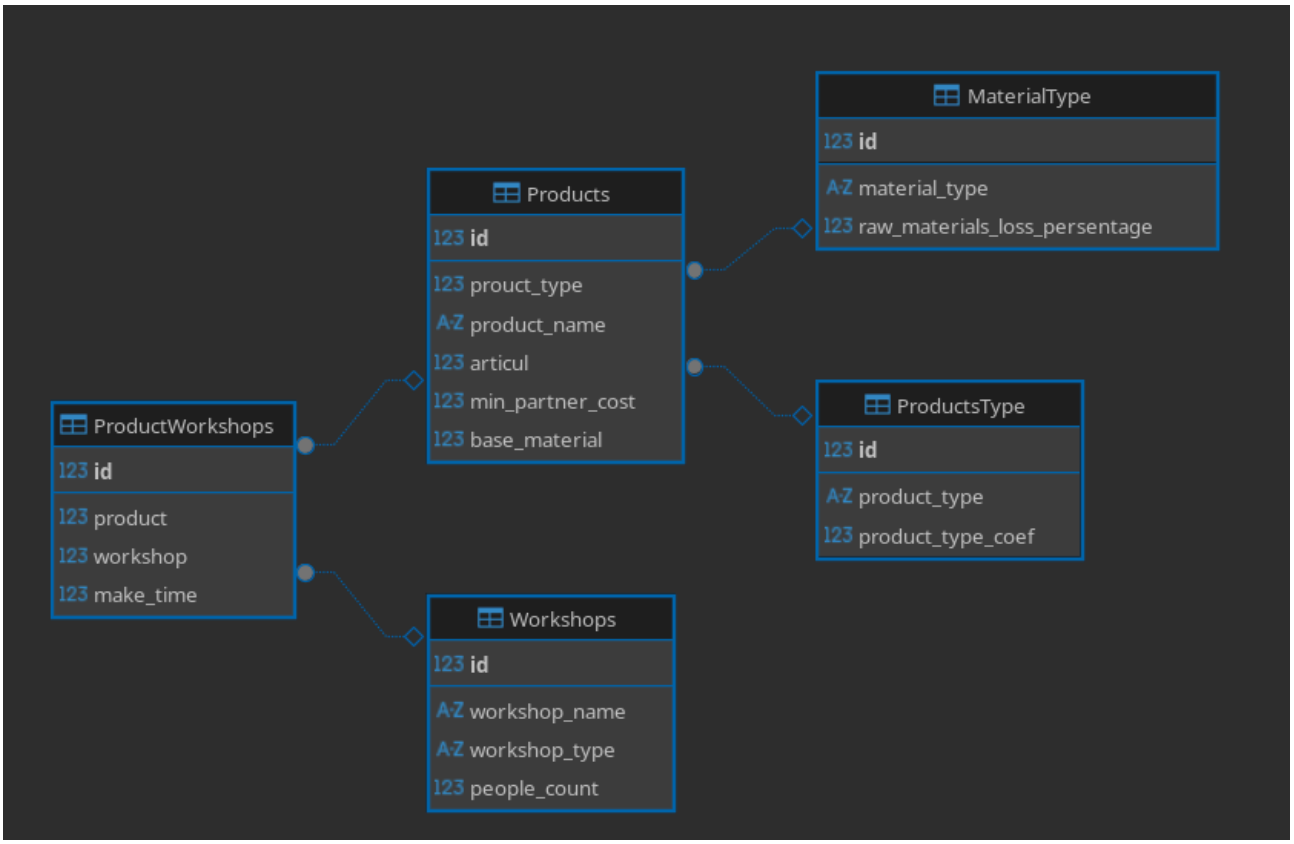

Рисунок 1.1 – ER-диаграмма

Запросы создания и заполнения базы данных представлены в приложении А.

1.4 Практическая реализация решения

Перед разработкой программного продукта была составлена блок-схема основного алгоритма работы системы. На данной схеме отражена последовательность действий пользователя при входе в приложение. Система отображает главную страницу. Далее система ожидает ввод. Если пользователь нажал на «добавить запись», то отрисовывается экран создания записи. Если «изменить запись», то окно изменения записи. Если просмотра цехов, то страница просмотра цехов. Также есть еще логика при работе в окнах создания и изменения. Блок-схема позволяет наглядно представить общую логику функционирования приложения и порядок перехода между основными этапами работы. См. на рисунке 1.2

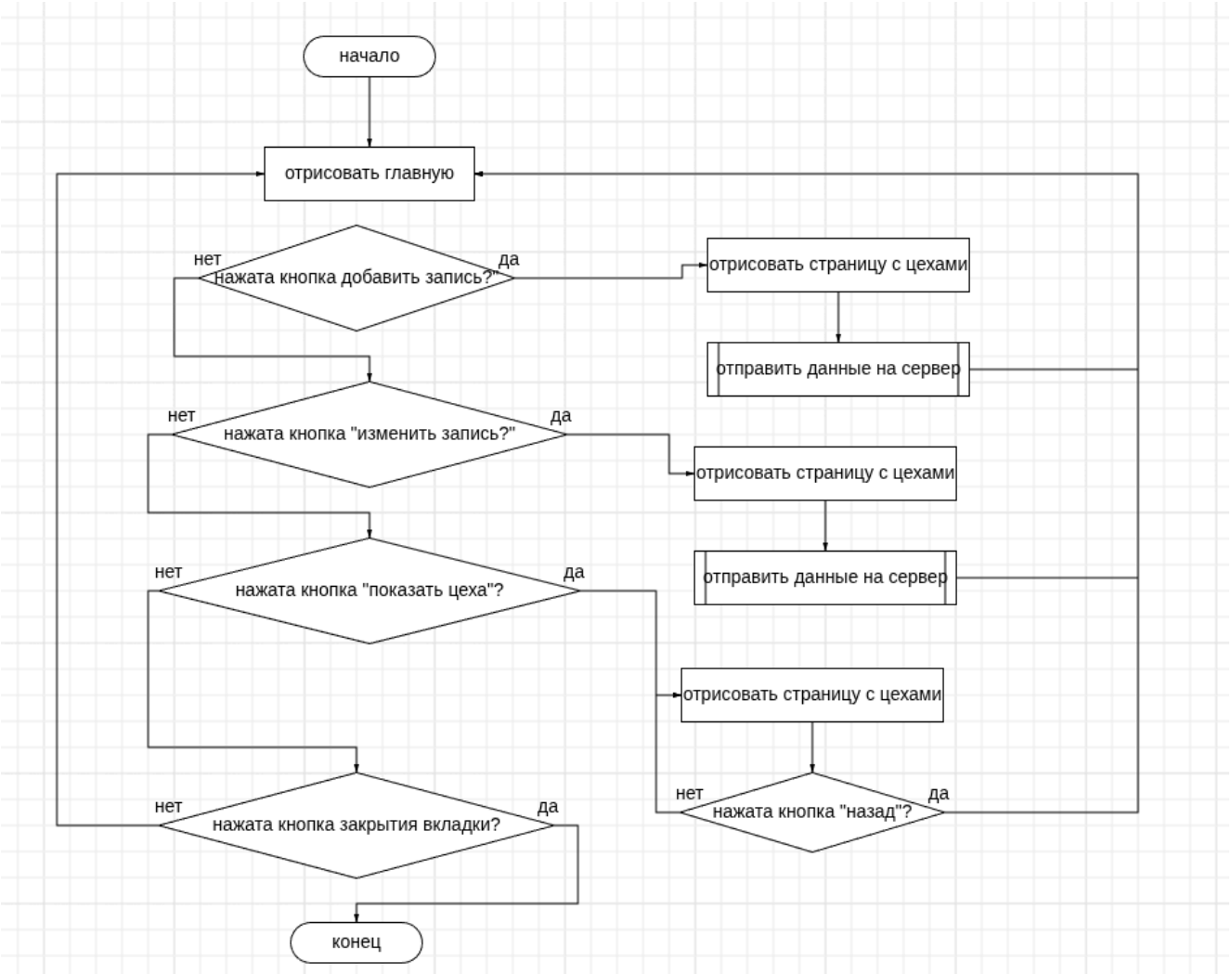

Рисунок 1.2 – Основной алгоритм работы системы

Программное обеспечение реализовано в виде веб-приложения на фреймворке fastapi с использованием технологии MVC и jinja2-представлений, что позволяет разделить структуру проекта на модели данных, компоненты доступа к базе данных, контроллеры, пользовательский интерфейс и служебные сервисы.

Таким образом, практическая реализация программы охватывает все основные функции системы учета продукции мебельной компании и обеспечивает работу пользователей в соответствии с их ролями и полномочиями.

1.5 Интерфейс программы

В процессе разработки программы было реализовано следующее:

- окно главной страницы с информацией о товарах и окно с детальной информацией о цехах по товару;
- окна редактирования информации о товарах и создания новой.

Интерфейс программы представлен на рисунках 1.3-1.6.

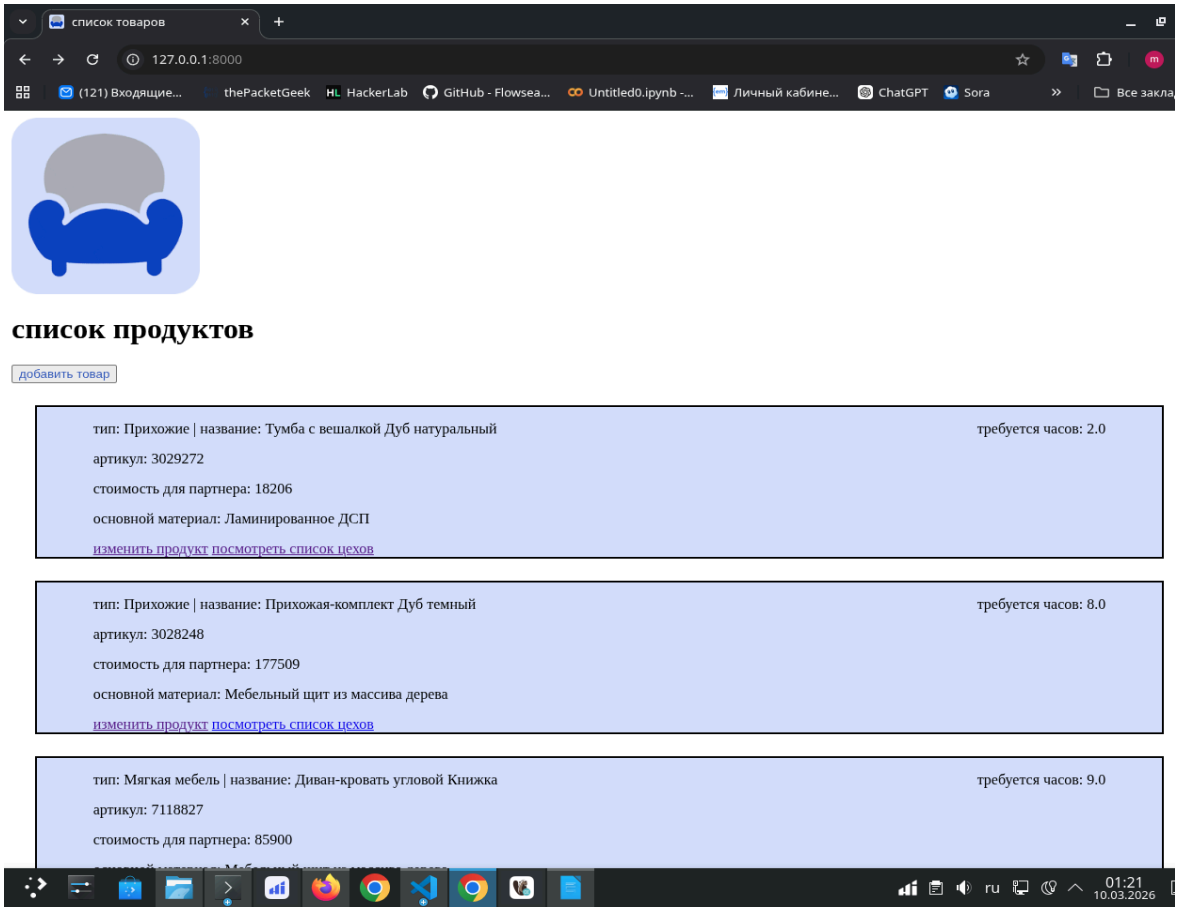

Рисунок 1.3 – Главная страница

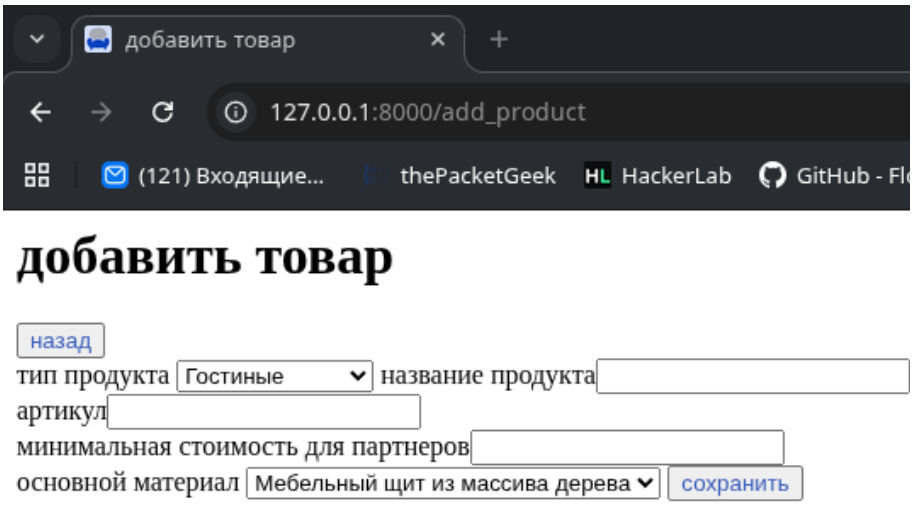

Рисунок 1.4 – Страница добавления товара

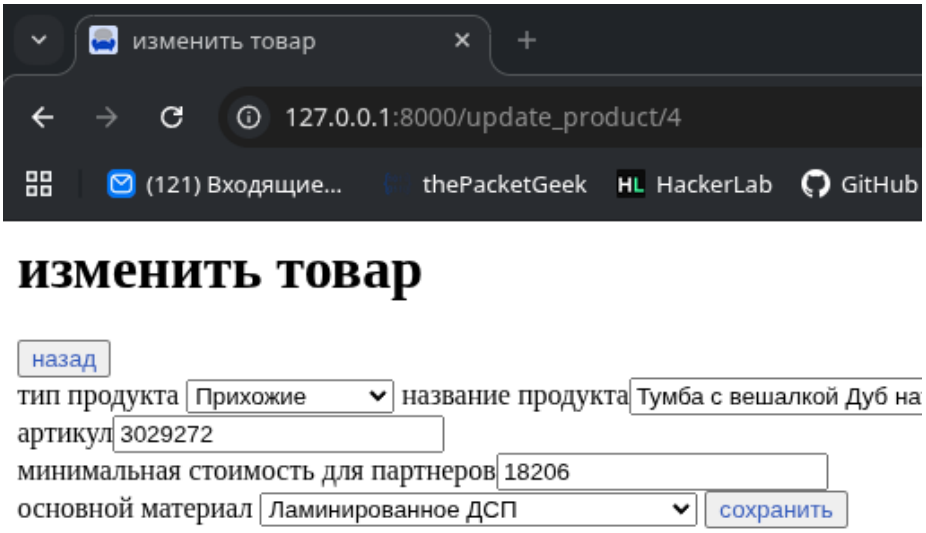

Рисунок 1.5 – Страница изменения товара

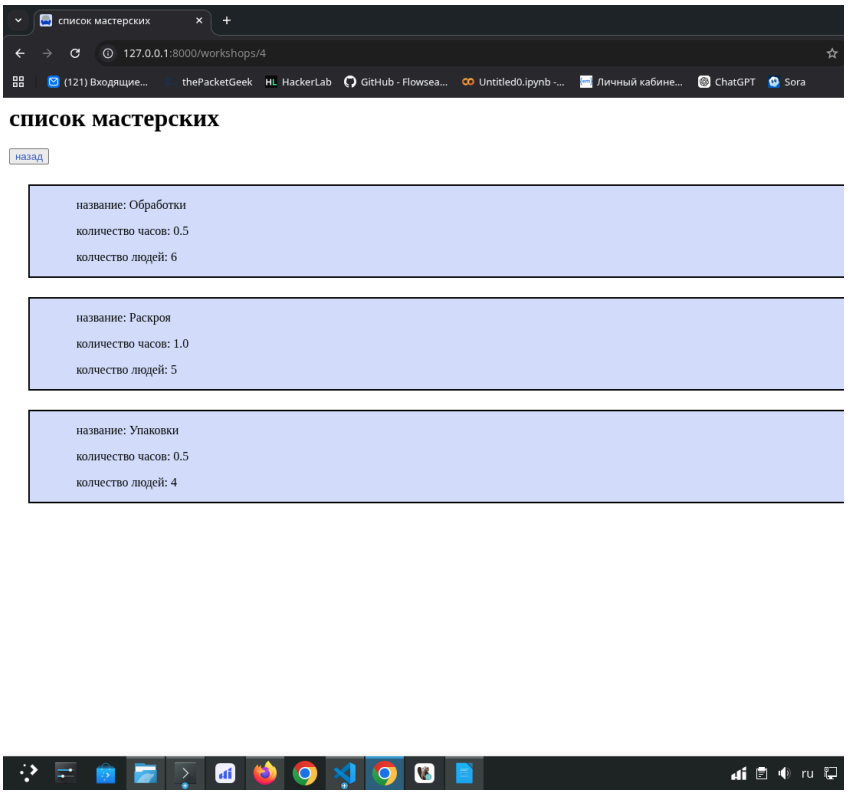

Рисунок 1.6 – Страница детальной информации о цехах

Метод определения количества сырья реализован и помещен по пути /demodemoex/main.py и представлен на рисунке 1.7.

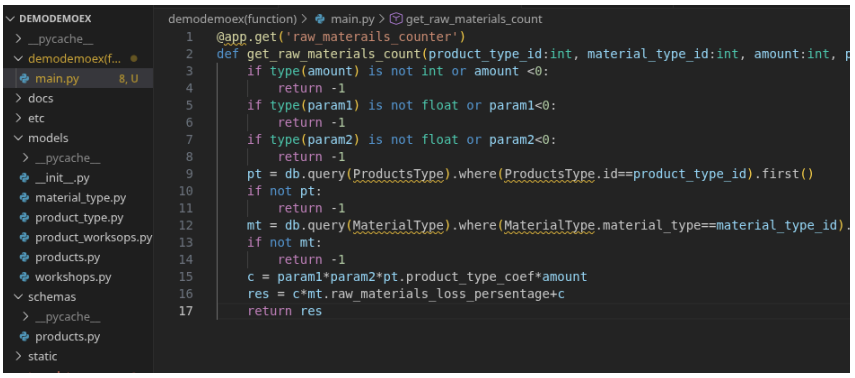

Рисунок 1.7 – Реализация метода расчета сырья

Валидация полей присутствует и представлена на рисунке 1.8

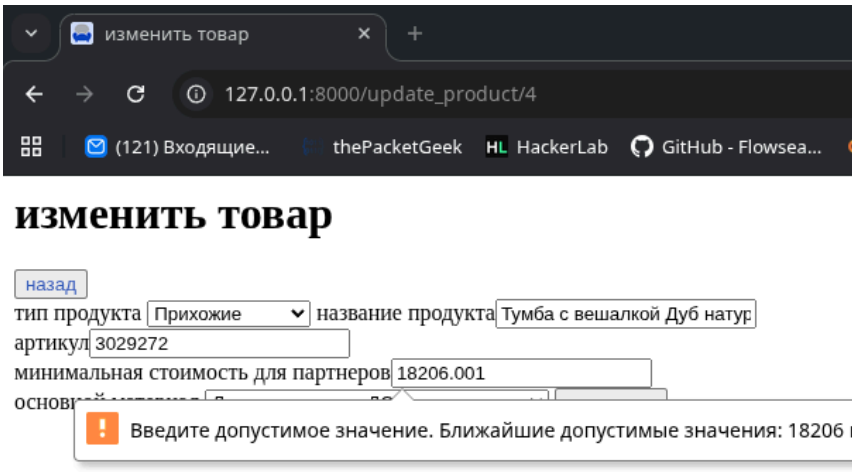

Рисунок 1.8 – Валидация данных

Алгоритм для работы с продукцией компании оформлен в виде функции get_hours_by_id и располагается в файле main.py. Представляет из себя обычный sql запрос. Представлен на рисунке 1.9.

```
50  def get_hours_by_id(product_id:int, db):
51      stmt = text('select sum(make_time) from "ProductWorkshops" where product=:pr
52      result = db.execute(stmt, {"product_id":product_id})
53      return result.scalar()
```

Рисунок 1.9 – Алгоритм для работы с продукцией компании

Реализована обработка ошибок, возвращающая статус коды со стороны сервера и информирование пользователя со стороны фронтенда. При том в задании было написано, что необходимо в случае чего реализовать информирование о невозвратных опасных действиях, явно подразумевая удаление записей, но явно это не было указано, поэтомму функционал удаления НЕ РЕАЛИЗОВАН. Ровно как и функционал добавления записей в смежную таблицу products-workshops.

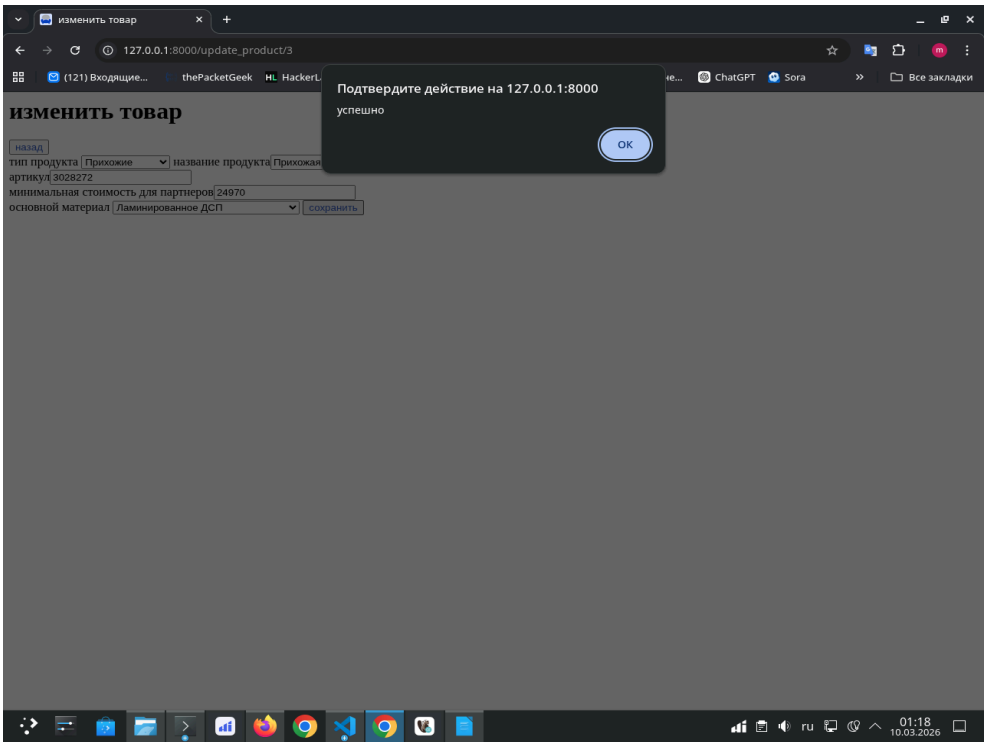

Рисунок 1.9 – Информирование об ошибках

2 Дипломный проект

2.1 Структура приложения

Приложение является сервисом, который управляет телефонными станциями и хранит конфигурации в отказоустойчивом хранилище данных. Соответственно, кластер должен состоять из:

1. Frontend (vue.js);
2. Backend сервер (fastapi);
3. Asterisk сервер – является хранилищем экземпляров asterisk;
4. База данных (mySQL) – используется для хранения метаданных asterisk (история звонков, адреса asterisk, пользователи sip, активные звонки и тд.);

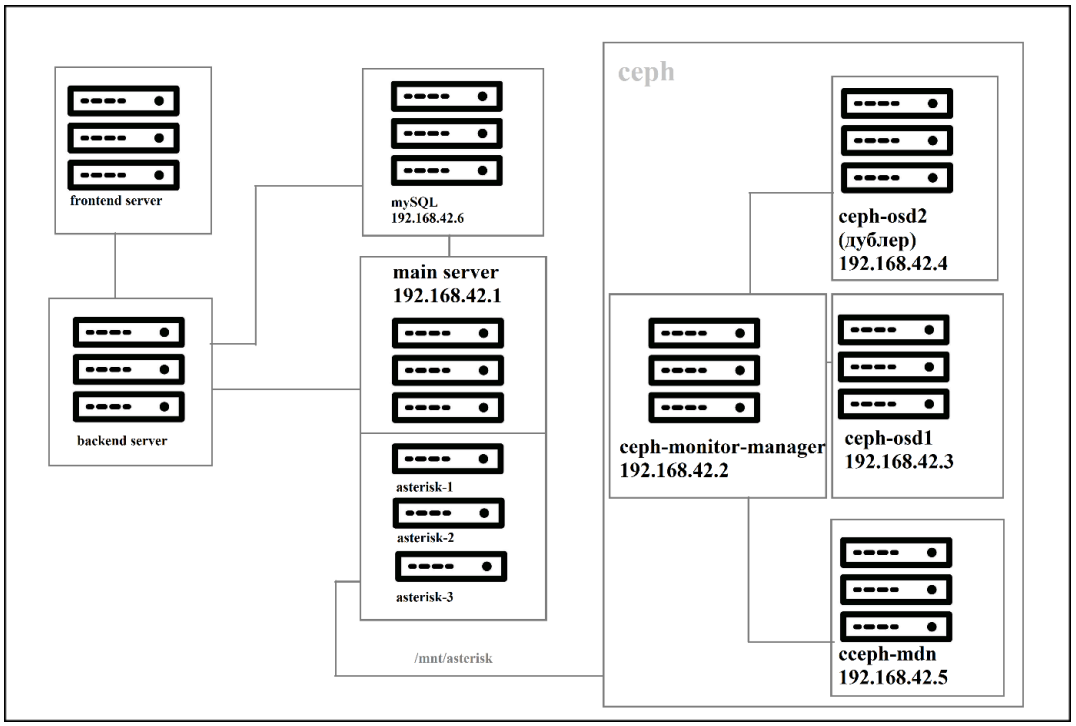

Рисунок 2.1 – Структура приложения

На рисунке 2.1 представлена структура всего приложения со всеми серверами, на которых будут развернуты приложения и хранилища, а также их взаимодействие друг с другом.

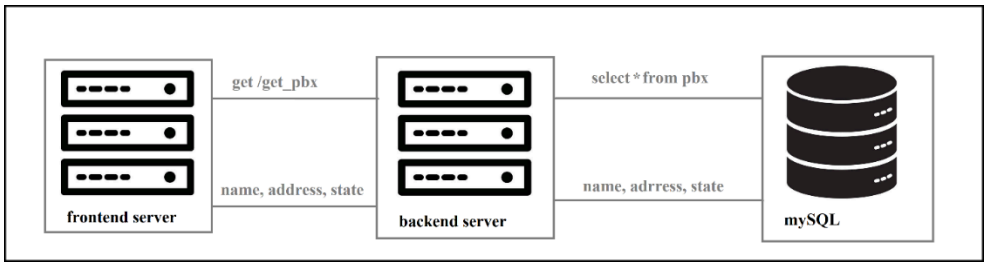

Рисунок 2.2 – Взаимодействие элементов приложения

На рисунке 2.2 представлена схема get запроса на получение экземпляров asterisk.

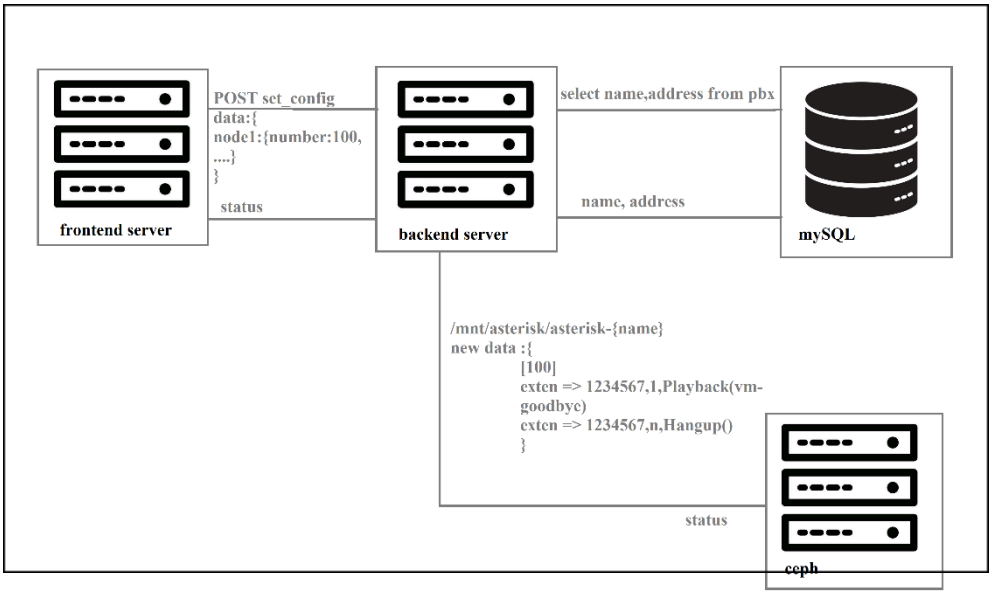

Рисунок 2.3 – Схема работы хранилища

На рисунке 2.3 представлена схема POST запроса на запись конфигурационного файла. Сначала из БД получается адрес нужной АТС и по нему обращается к ceph и записывается в нужный файл на примонтированной директории.

2.2 Макеты основных страниц приложения

На макетах можно увидеть предварительное представление страниц веб-приложение. Макеты помогают в разработке, так как позволяют определить

единый стиль для всех страниц, а также ускорят написание кода так как визуальное представление требуемых результатов всегда находится перед глазами.

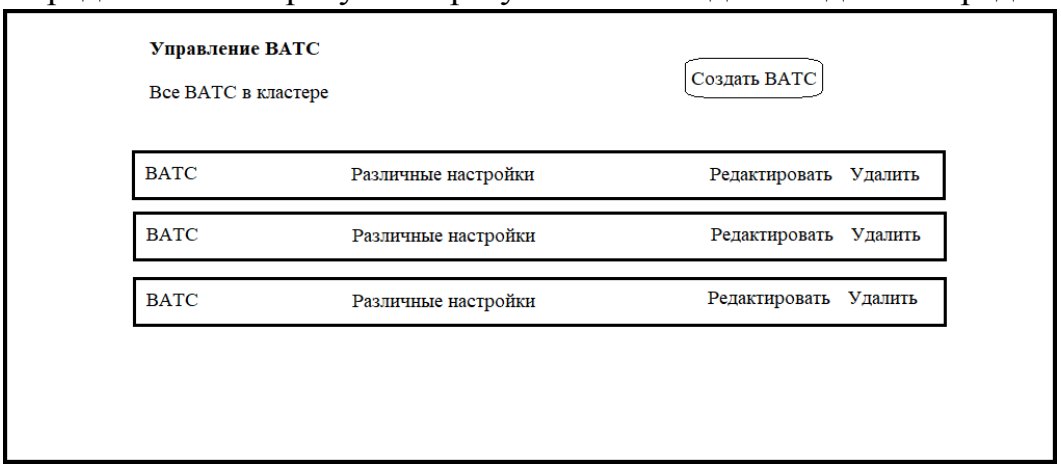

Рисунок 2.4 – Страница управления ВАТС

На данном на рисунке 2.4 макете страницы предполагается возможность наблюдать все ВАТС, редактировать и удалять их.

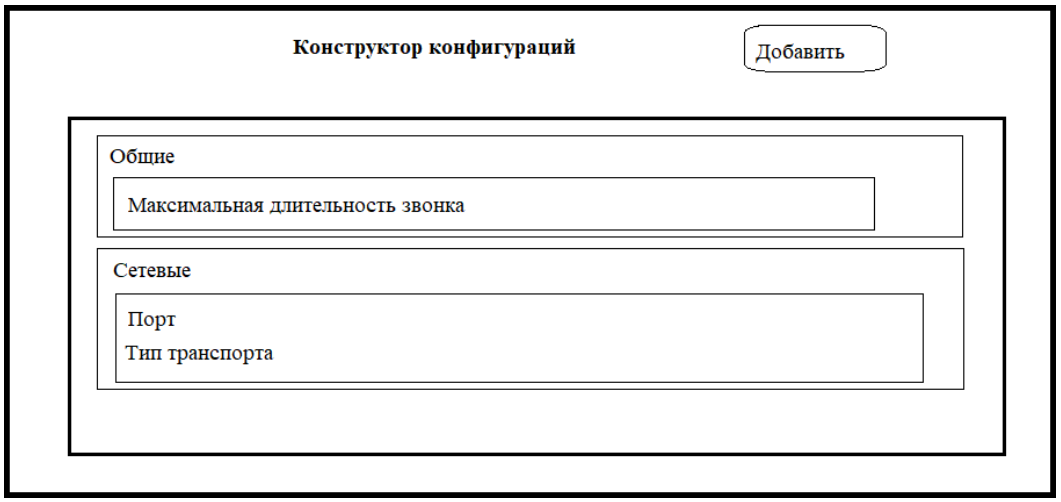

Рисунок 2.5 – Окно редактирования ВАТС

На данном на рисунке 2.5 макете модального окна предполагается возможность создавать и редактировать конфигурации ВАТС.

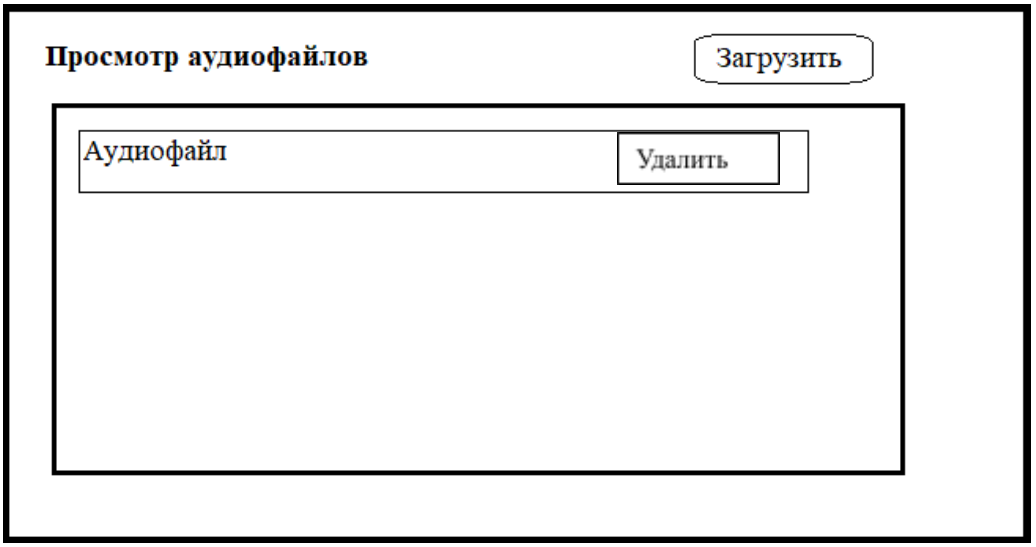

Рисунок 2.6 – Страница аудиофайлов

На данном на рисунке 2.6 макете страницы предполагается возможность просматривать, удалять и загружать аудиофайлы.

2.3 Разработка API

Для компании необходимым условием приложения было создание backend сервера, который бы создавал экземпляры asterisk(до 200 штук), изменял их конфигурации, удалял. Было несколько подходов к реализации данной логики приложения:

1 С помощью docker. Сервер создает docker-compose.yaml файл и с помощью docker-compose создает и запускает docker-контейнер. Docker контейнер хранит свои конфигурационные файлы в volume, доступным из гостевой системы. В дальнейшем volume интегрируется с ceph.

2 Нативно. С помощью bash скрипта создается экземпляр asterisk на определенном порту. Доступ к конфигурациям осуществляется через sshmount - примонтирование директории конфигураций к backend - серверу. Позже этот mount заменяется на mount к ceph.

Решено было выбрать первый подход за счет его модульности и простоты развертывания.

Также оба способа предполагают наличие базы данных для хранения метаданных, в частности имени АТС и портов, на которых они располагаются. Заказчик явно указал в требованиях использование mySQL.

Для backend сервера необходимо реализовать роуты для создания, удаления АТС и изменения их конфигураций(в том числе и маршрутизации звонков).

GET /instances/ - получить список АТС.

Пример использования на рисунке 2.12;

POST /instances/ - создать АТС с параметрами name, sip_port, http_port;

GET /instances/{id}/config/{config_name} - получить содержимое конфигурационного файла config_name. Пример использования представлен на рисунке 2.7;

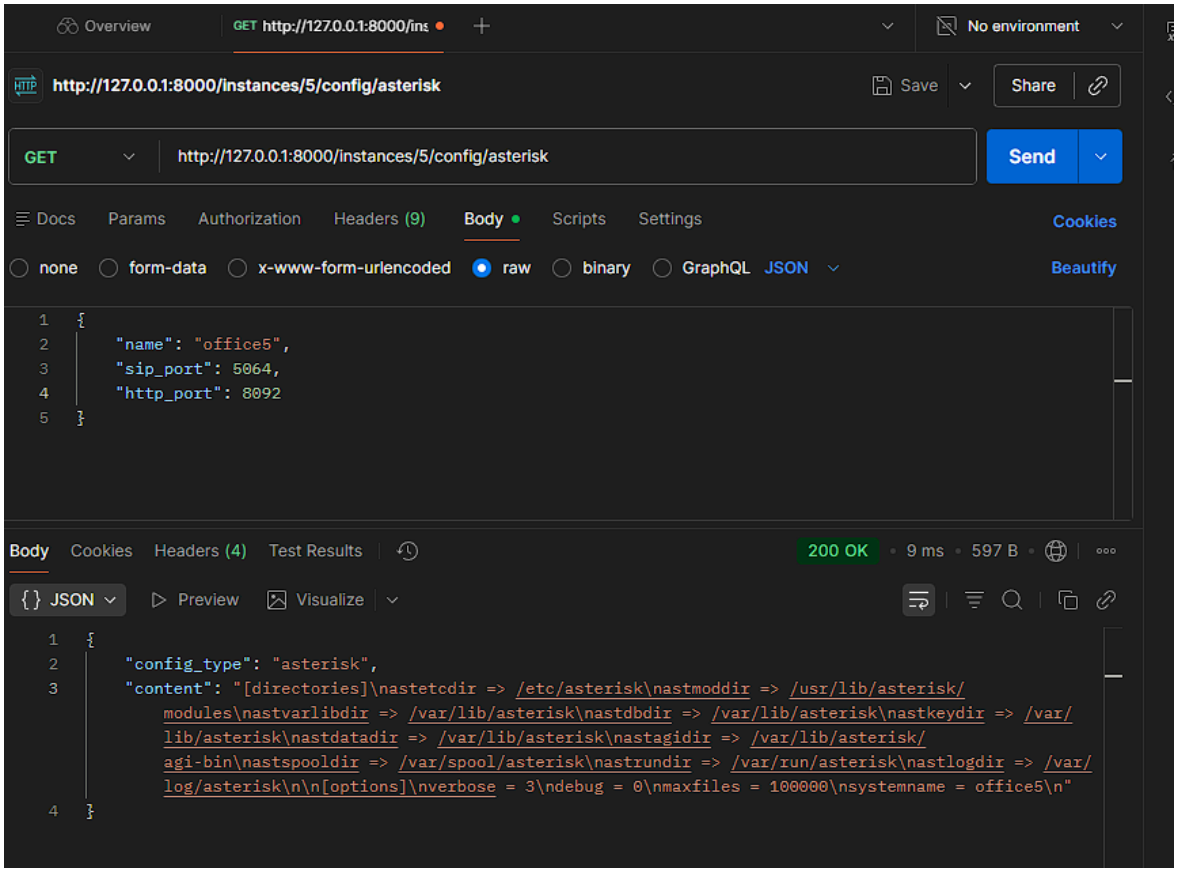

Рисунок 2.7 - Реализация get запроса, получающего информацию по конфигурационным файлам

PUT /instances/{id}/config/{config_name} - дополнить конфигурационный файл config_name. Пример использования представлен на рисунке 2.8;

POST /instances/{id}/config/{config_name} - заменить содержимое конфигурационного файла config_name;

POST /instances/{id}/reload – перезагрузить экземпляр asterisk;

GET /instances/{id}/users – получить список sip-пользователей;

PUT /instances/{id}/users/{user_id} – обновить информацию о пользователе;
DELETE /instances/{id}/users/{user_id} – удалить sip-пользователя;
GET /cdr/ - получить список звонков;
GET /cdr/stats/ - получить детальную информацию по списку звонков;
GET /cdr/active/ - получить список активных звонков;

Также решено было выбрать фреймворк fastapi для разработки backend сервера за счет его простоты разработки и надежности.

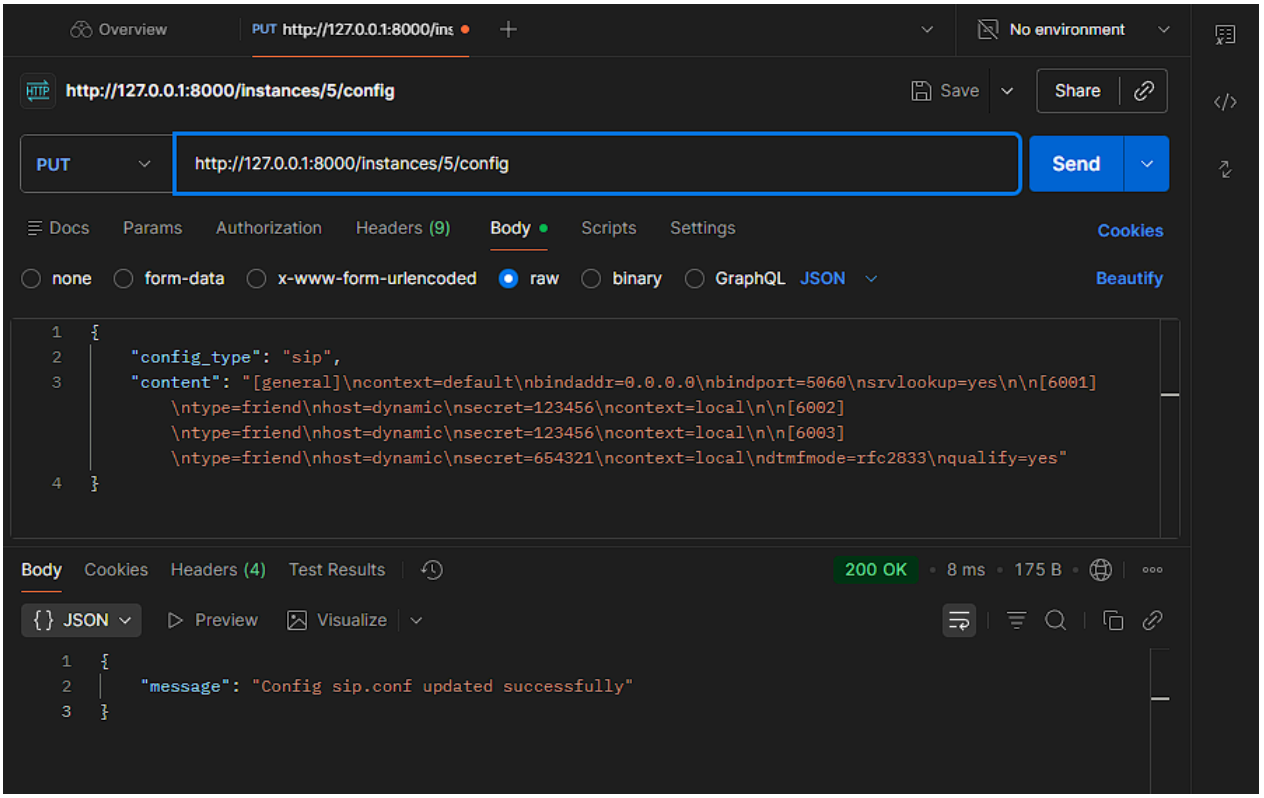

Рисунок 2.11 - Запрос на изменение конфигурационного файла

Логика создания АТС. На роут /instances/ приходит json с параметрами АТС. На его основе создается docker-compose.yaml файл, содержащий имя АТС и порты. После с помощью subprocess выполняется команда docker-compose run -d, запускающая контейнер с образом deploy/asterisk. Метаданные записываются в базу данных.

2.4 Инструкция по работе с продуктом

В данном разделе будут указаны основные функции, реализованные в веб-приложении по указанным требованиям, подробное описание всех страниц и работа с ними.

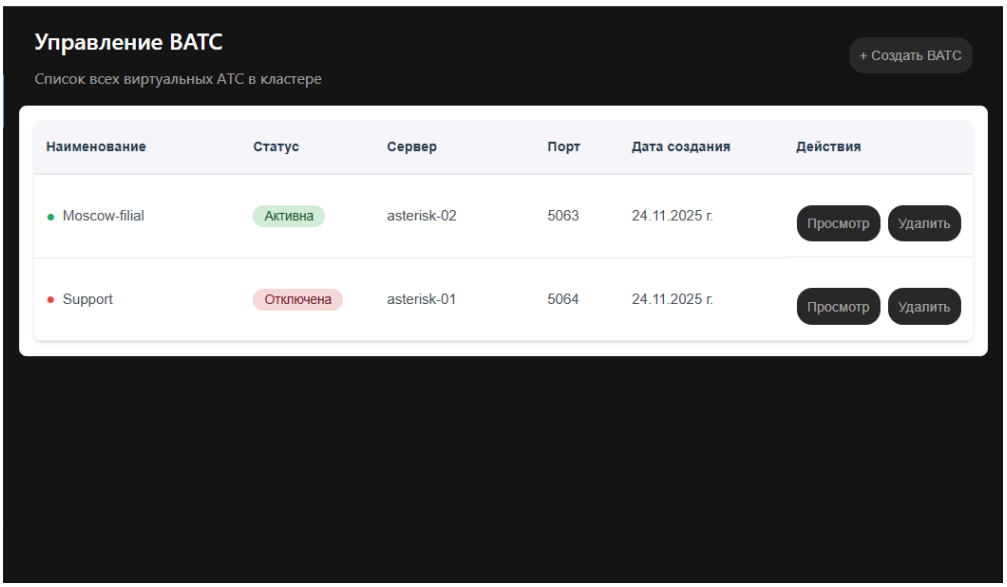

Рисунок 2.12 – Текущее состояние списка ВАТС

На рисунке 2.12 можно наблюдать список ВАТС, записанных в БД. С помощью кнопки «Создать ВАТС» можно создать новую ВАТС, после нажатия откроется модальное окно, в котором необходимо ввести данные о ВАТС, создание происходит в 2 этапа: ввод названия, а позже – ввод остальных настроек. Взаимодействие с ними происходит через кнопки «Просмотр», позволяющей увидеть конфигурацию текущей ВАТС и внести изменения через модальное окно, которое открывается при нажатии на кнопку. Кнопка «Удалить» удаляет ВАТС из списка.

Листинг этой страницы можно увидеть в приложении А.

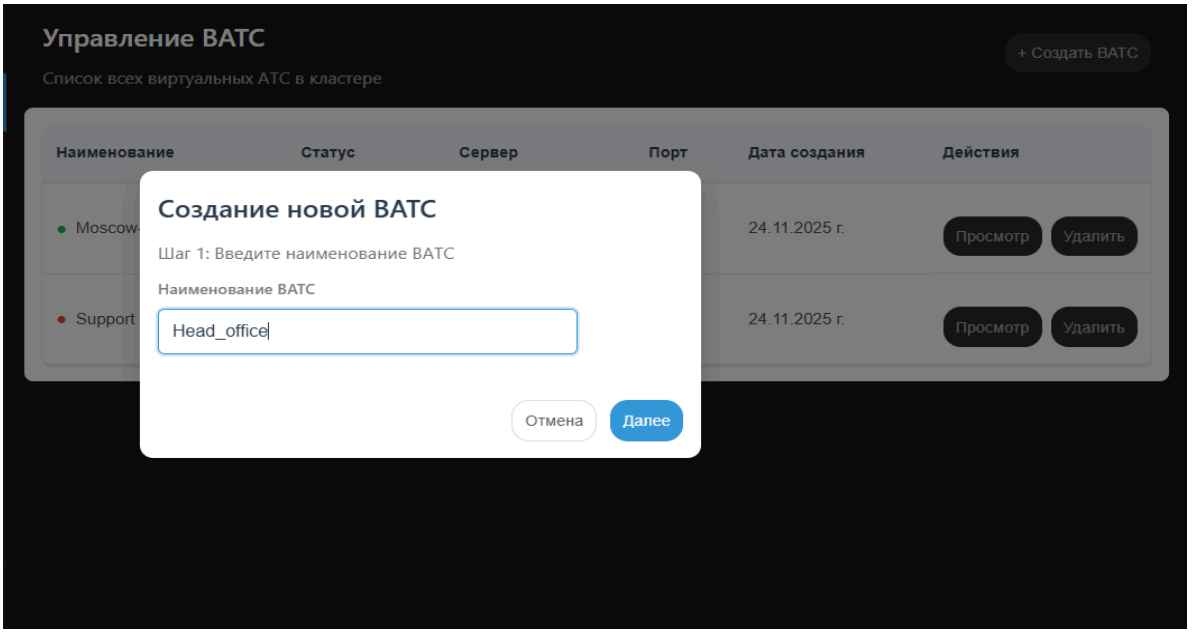

Рисунок 2.13 – Создание новой ВАТС

На рисунке 2.13 изображено начало создания новой ВАТС с названием «Head_office». На данном этапе вводится только название.

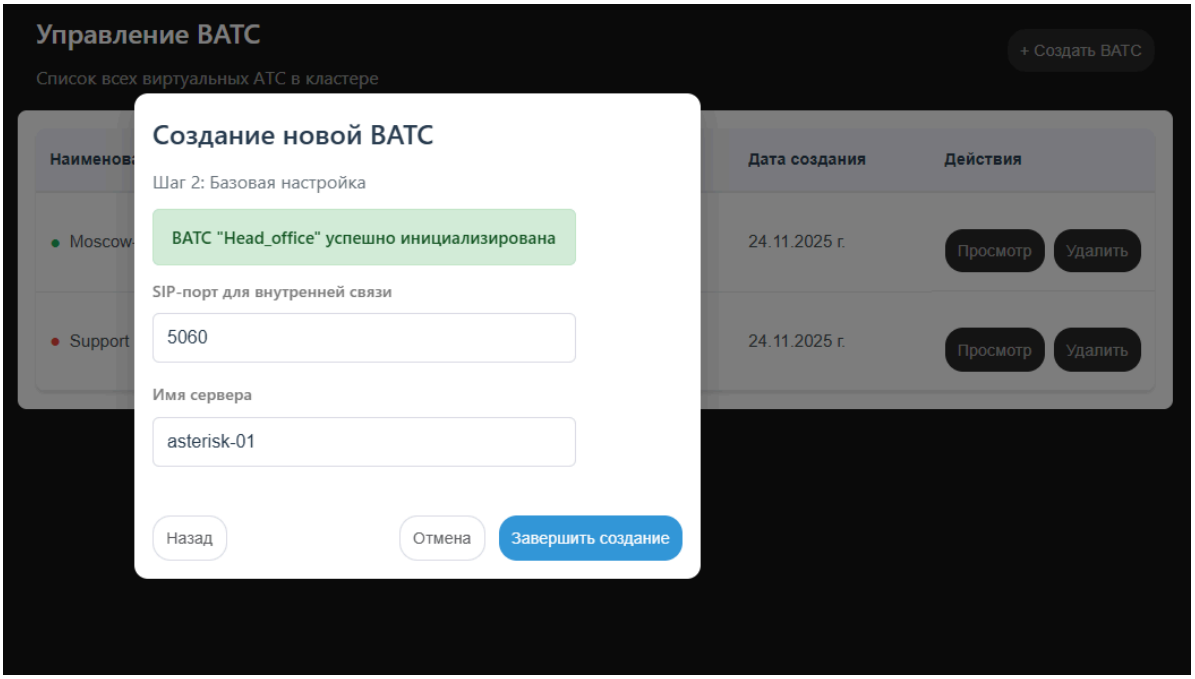

Рисунок 2.14 – Базовая настройка при создании ВАТС

На рисунке 2.14 указываются SIP-порт и имя сервера, на котором будет располагаться ВАТС, это является базовой настройкой при создании. После нажатия на кнопку «Завершить создание» отправляется post-запрос на сервер с созданием новой записи в таблице с ВАТС.

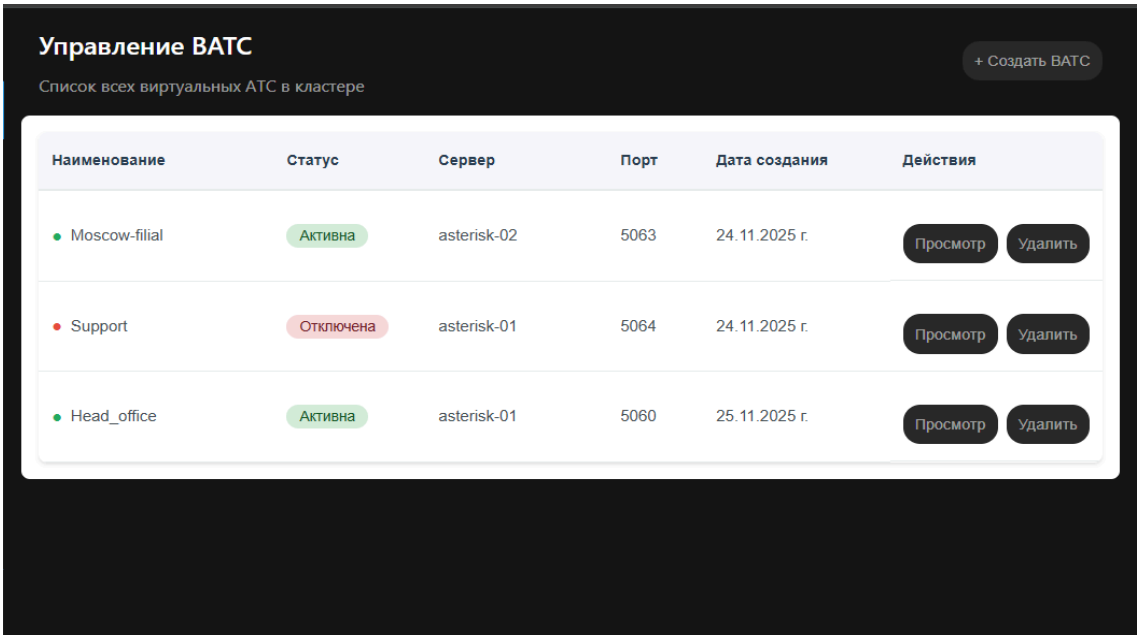

Рисунок 2.15 – Список ВАТС после добавления новой

На рисунке 2.15 можно наблюдать, что список ВАТС изменился: в нем появилась новая запись.

```
mysql> select * from asterisk_instances;
+----+--------------+---------+----------+-------------------------------+----------+---------------------+---------------------+
| id | name         | sip_port | http_port | config_path                   | status   | created_at          | updated_at          |
+----+--------------+---------+----------+-------------------------------+----------+---------------------+---------------------+
|  3 | Moscow-filial |    5063 |     8895 | ./asterisk_configs/Moscow-filial | creating | 2025-11-25 10:47:34 | 2025-11-25 10:47:34 |
|  4 | Support      |    5064 |     8896 | ./asterisk_configs/Support    | running  | 2025-11-25 11:18:57 | 2025-11-25 11:19:00 |
|  5 | Head_office  |    5065 |     8897 | ./asterisk_configs/Head_office | running  | 2025-11-25 11:20:03 | 2025-11-25 11:20:05 |
+----+--------------+---------+----------+-------------------------------+----------+---------------------+---------------------+
```

Рисунок 2.16 – Таблица ВАТС в БД

На рисунке 2.16 можно видеть, что записи в БД тоже обновились.

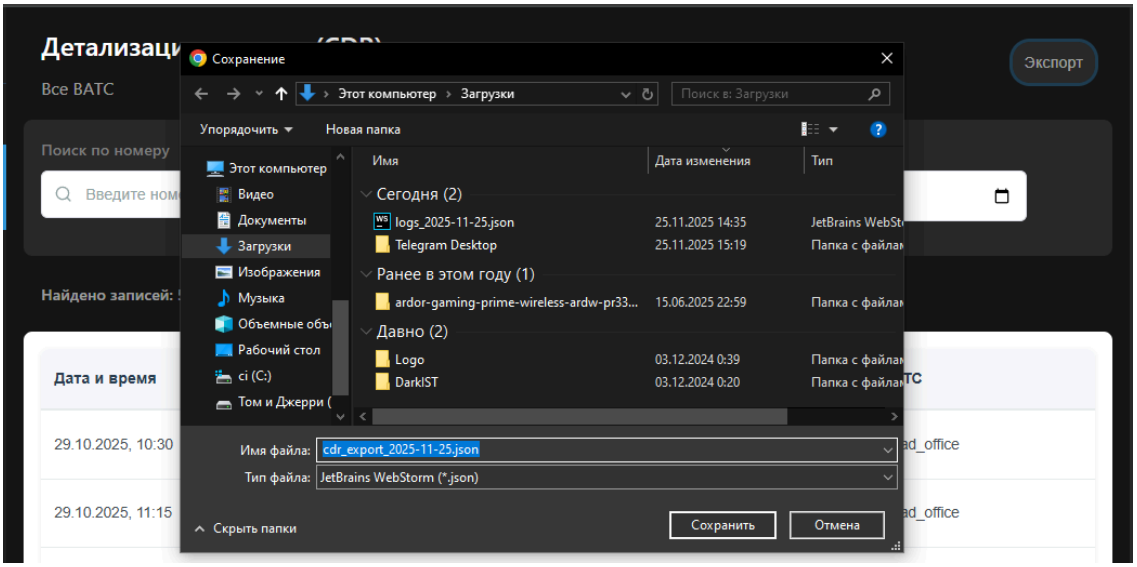

Рисунок 2.17 – Экспорт истории звонков

На рисунке 2.17 изображена установка истории звонков в формате JSON на локальное устройство, это происходит путем нажатия на кнопку «Экспорт» в правом верхнем углу страницы. Также на данной странице доступна фильтрация всех записей в таблице по номерам телефонов звонившего или адресата звонка, статусу звонка и дате звонка, фильтрация происходит с помощью полей ввода в верхней части страницы над таблицей с вызовами.

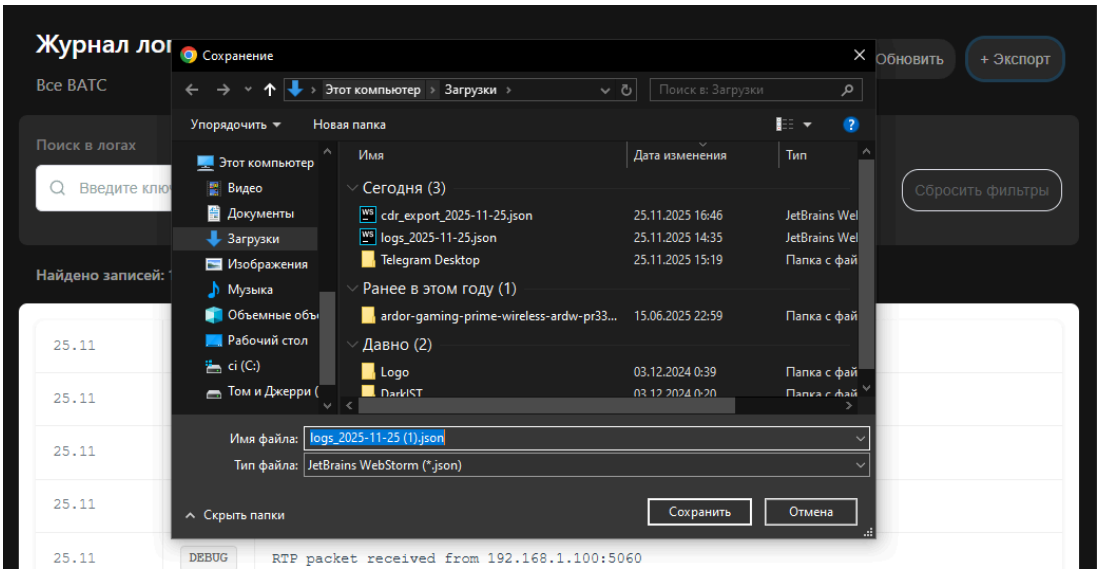

Рисунок 2.18 – Экспорт логов

На рисунке 2.18 изображена установка логов в формате JSON на локальное устройство, это происходит путем нажатия на кнопку «Экспорт» в правом верхнем углу страницы. На этой странице изображена история логов, появляющихся при выполнении каких-либо действий со стороны сервера. Также на данной странице доступна фильтрация записей в таблице по ключевым словам, содержащимся в логах и их уровню. Предусмотрена и кнопка «Обновить» обновляющая содержимое таблицы, что позволяет отслеживать логи в реальном времени.

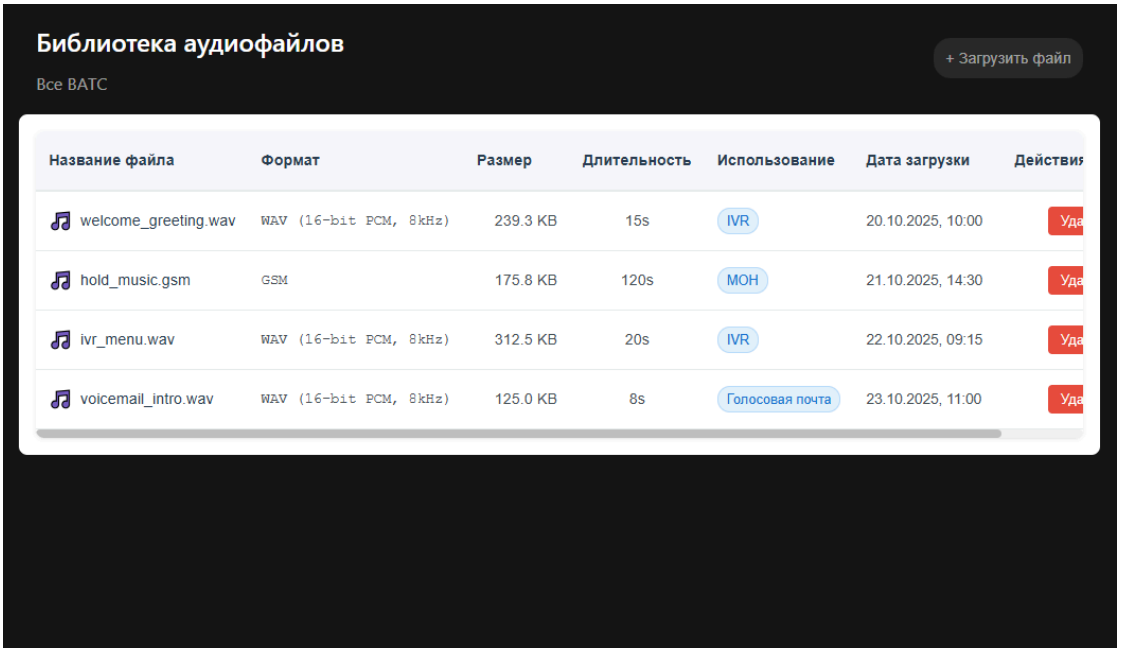

Рисунок 2.19 – Текущее состояние библиотеки аудиофайлов

На рисунке 2.19 можно увидеть таблицу аудиофайлов, загруженных на сервер,

также можно загрузить новые через кнопку «Загрузить файл». Предусмотрена возможность удалить аудиофайл по какой бы то ни было причине путем нажатия кнопки «Удалить» в таблице, после нажатия на кнопку будет удален аудиофайл, напротив которого в таблице находилась кнопка.

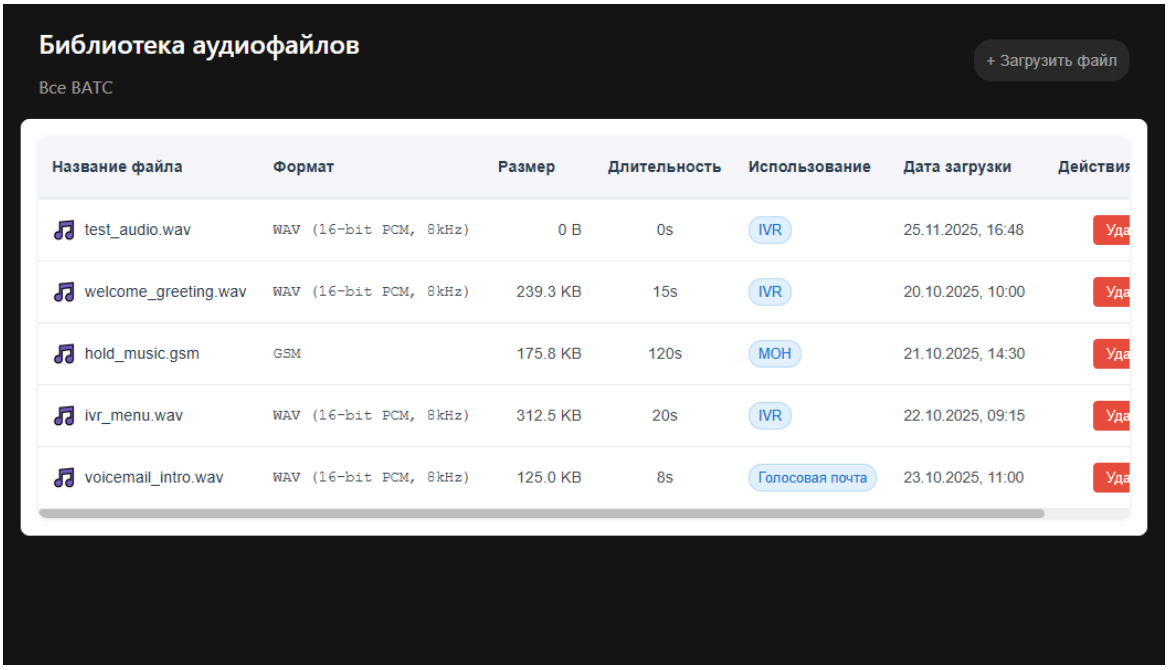

Рисунок 2.20 – Обновление таблицы

На рисунке 2.20 можно увидеть состояние таблицы после добавления нового аудиофайла.

Заключение

Подводя итоги учебной практики можно сказать, что поставленные цели и задачи выполнены, приложение работает корректно, заказчик будет доволен. Приложение позволяет просматривать информацию о товарах, информацию о цехах, высчитывать общее время создания продукта, создавать новые записи о продуктах и редактировать уже созданные.

Подводя итоги дипломного проекта, можно сказать, что поставленные цели и задачи выполнены, приложение корректно работает и готово к использованию или дальнейшей доработке. Приложение позволит предприятию автоматизировать процессы, которые раньше производились вручную, это ускорит работу с ВАТС, позволит сотрудникам уделять больше времени другим более срочным и важным задачам, уменьшит на них нагрузку.

Поставленная тема многогранная и интересная, так как используется множество незнакомых технологий и систем, также позволяет расширить знания касательно уже знакомых технологий ввиду необходимости создания собственного решения поставленной проблемы, были изучены новые темы и расширен кругозор. Курсовой проект можно считать успешно выполненным и в полной мере реализованным.

Библиография

1 FastAPI: Official Documentation [Электронный ресурс] – Режим доступа: https://fastapi.tiangolo.com/

2 Vue.js: The Progressive JavaScript Framework [Электронный ресурс] – Режим доступа: https://vuejs.org/guide/introduction.html

3 Шаблон приложения FastAPI + ... [Электронный ресурс] – Режим доступа: https://github.com/profcomff/app-template

4 FastAPI + Vue.js: Создаем веб-проект (Урок 7) [Электронный ресурс] – Режим доступа: https://icoder.uz/veb-razrabotka/wibc9nunjfu-fastapi-tutorial-7-web-project/

Приложение А
(справочное)
Листинг программы

Листинг SQL запросов к бд

```
create database main;
create table MaterialType (
    id serial primary key,
    material_type text,
    raw_materials_loss_persentage decimal
);
create table ProductsType(
    id serial primary key,
    product_type text,
    product_type_coef deciaml,
);
create table Workshops(
    id serial primary key,
    workshop_name text,
    workshop_type text,
    people_count int
);
create table Products(
    id serial primary key,
    product_type int references "Products"(id)
    product_name text,
    articul int,
    min_partner_cost decimal,
    base_material int references "MaterialType"(id)
);
create table ProductWorkshops(
    id serial primary key,
    products int references "Products"(id),
    workshop int references "Workshops"(id),
    make_time decimal
);
```